

Class 07 — Student Notes

Bong Study Hub — Foundation Batch 2026

Operators & Expressions

Class 01's LOGIC, Made Literal

These notes summarize what we covered in class. Use them to revise, not to learn the topic for the first time — the real learning happened at the keyboard, predicting each result before running it. No `if`, no loops, and no `input()` appear here — those come in later classes.

The Big Question

Class 06 ended with a question we didn't answer: we can print numbers, but we haven't done any math with them.

"Now that we can store values, how do we actually compute, compare, and combine them?"

Quick connection to Class 01

Class 01's very first chain included a box we never actually opened:

PROBLEM → LOGIC → ALGORITHM → PROGRAM → RESULT

LOGIC meant "reasoning about how to solve the problem" — but we never showed what that reasoning looks like in code. Today, it does:

LOGIC (Class 01) → **OPERATORS & EXPRESSIONS** (today)

PROBLEM
↓
LOGIC
↓
ALGORITHM
↓
PYTHON CODE
↓
VARIABLES
↓
EXPRESSIONS
↓
RUN
↓
OUTPUT / RESULT

1. What Is an Operator?

An **operator** is a symbol that does something with one or more values. If variables are the nouns of programming, operators are the **verbs** — they act on values and hand you back a new one.

The values an operator acts on are called its **operands**. In `price1 + price2`, `price1` and `price2` are the operands; `+` is the operator.

Today we meet three families:

- **Arithmetic** operators — do math.
- **Comparison** operators — ask true/false questions.
- **Logical** operators — combine true/false answers.

2. Arithmetic Operators: The Basics

```
price1 = 50
price2 = 30
total = price1 + price2
print(total)
```

→ 80

This is genuinely new: a variable can now hold the **result of a computation**, not just something typed in directly.

```
print(10 - 3) # subtraction → 7
print(4 * 5) # multiplication → 20
print(9 / 2) # division → 4.5
```

Important: / always gives you a float result, even when the numbers divide evenly. `10 / 2` gives `5.0`, not `5`.

3. Two Special Operators: // and %

```
print(9 // 2) # floor division → 4
print(9 % 2) # modulus → 1
```

// (floor division) divides, then keeps only the whole-number part, dropping anything after the decimal point — `9 / 2` is `4.5`, floored down to `4`.

% (modulus) gives you the **remainder** left over — `9` is 4 groups of `2` with `1` left over.

`number % 2 == 0` is exactly how programs check whether a number is even — we're not writing that check yet (that needs an `if`, coming in Class 08), but now you know where the tool comes from.

4. Order of Operations

```
print(2 + 3 * 4)
```

→ 14, not 20 — multiplication happens before addition, exactly like math class.

```
print((2 + 3) * 4)
```

→ 20 — parentheses force that part to happen first.

When in doubt, use parentheses to make your intent explicit, even where they aren't strictly required.

5. Expressions: Building Bigger Values

Every line we've written today — `price1 + price2`, `9 // 2`, `(2 + 3) * 4` — is an **expression**: any piece of code that Python can evaluate down to a single value. A plain value like `18` is the simplest possible expression.

```
subtotal = price1 + price2
tax = subtotal * 0.1
total = subtotal + tax
print(total)
```

Each line's expression can use the result of a previous line — that's how real programs build up complicated calculations, one small, understandable expression at a time.

6. Comparison Operators

```
age = 18
print(age == 18) # True
print(age > 21) # False
print(age != 20) # True
print(age <= 18) # True
```

The full set:

```
== equal to
!= not equal to
< less than
> greater than
<= less than or equal to
>= greater than or equal to
```

Every comparison produces a **boolean** — exactly the data type from Class 06. Comparison operators are literally where booleans come from in a real program.

7. A Common Trap: = vs ==

```
age = 18
age == 18
```

One = stores — `age = 18` means "put 18 into age." **Two == asks a question** — `age == 18` means "is age equal to 18?" and hands back `True` or `False`. They look almost identical and mean completely different things.

This mix-up is so common that even experienced programmers still catch themselves doing it. If your code isn't behaving the way you expect, checking `=` vs. `==` is one of the first things to check.

8. Logical Operators: and, or, not

```
age = 16
print(age >= 13 and age <= 19) # True
print(age < 13 or age > 19) # False
print(not (age == 16)) # False
```

- `and` gives `True` only if **both** sides are true.
- `or` gives `True` if **at least one** side is true.
- `not` flips a boolean — `True` becomes `False` and vice versa.

Plain-language translations:

```
age >= 13 and age <= 19 → "age is at least 13 AND at most 19"
age < 13 or age > 19 → "age is younger than 13 OR older than 19"
not (age == 16) → "age is NOT 16"
```

9. Combining Comparisons with Logical Operators

```
is_weekend = True
is_raining = False
print(is_weekend and not is_raining)
```

This reads almost like English: "is it the weekend, and is it *not* raining?" That's a real rule a program might check — built entirely from variables and operators.

10. Connecting Back to Class 01

Every time you've ever reasoned "this has to be true AND that has to be true," or "check whether this equals that," you were doing exactly what today's operators do.

Arithmetic, comparison, and logical operators are reasoning, written down precisely enough for a computer to carry out. They're Class 01's LOGIC box, made literal.

Try It Yourself — Level Up Your Profile Card

Open your Variable Profile Card from Class 06 (or rebuild it quickly). Add:

- **One arithmetic expression**, using an existing number variable — e.g. `age_in_months = age * 12`.
- **One comparison**, producing a boolean — e.g. `is_adult = age >= 18`.
- **One logical expression**, combining two comparisons — e.g. `is_teenager = age >= 13 and age <= 19`.

Print all three new results. Before you run each line, guess what it will show.

Quick Recap

- An **operator** is a symbol that acts on values (**operands**) and produces a new one.
- Arithmetic operators — `+` `-` `*` `/` — compute. `/` always gives a float.
- `//` keeps only the whole-number part of division; `%` gives the remainder.
- **Order of operations** matches math class: `*` `/` `//` `%` before `+` `-`, unless parentheses say otherwise.
- An **expression** is any piece of code that produces a value.

- **Comparison operators** — `==` `!=` `<` `>` `<=` `>=` — always produce a **boolean**.
- `=` stores a value. `==` asks a question. Never the same thing.
- **Logical operators** — `and`, `or`, `not` — combine booleans into one boolean answer.
- Today's operators are Class 01's **LOGIC**, made literal.

Key Terms

Term	Meaning
Operator	A symbol that does something with one or more values.
Operand	A value an operator acts on.
Expression	Any piece of code that produces a value.
Arithmetic operator	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>//</code> <code>%</code> — operators that do math.
Floor division (<code>//</code>)	Division that keeps only the whole-number part.
Modulus (<code>%</code>)	The remainder left over after division.
Comparison operator	<code>==</code> <code>!=</code> <code><</code> <code>></code> <code><=</code> <code>>=</code> — asks a true/false question, producing a boolean.
Logical operator	<code>and</code> , <code>or</code> , <code>not</code> — combines boolean values into one boolean answer.
Order of operations	The fixed order Python uses to evaluate an expression with multiple operators.

Think About It

Try answering these in your own words — you don't need exact wording, just the right idea.

1. What's the difference between `/` and `//`?
2. What does `%` give you?
3. What data type does a comparison always produce?
4. What's the difference between `=` and `==`?
5. When does `and` give `True`? When does `or` give `True`?
6. How does this class connect back to Class 01's **LOGIC** box?

Final Takeaway

"For six classes, LOGIC was just a word on a board. Today, it has a shape: arithmetic operators that compute, comparison operators that ask true/false questions, and logical operators that combine those answers. Every rule you will ever teach a computer to follow starts with exactly these tools."

Right now, every comparison and every logical result just gets printed — it doesn't actually change what the program does next. What if a program needed to behave differently depending on the answer? That's exactly where we pick up next class.